Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
O'Reilly Media 2004

Design Pattern: other patterns

Francesco Guerra
francesco.guerra@unimore.it

Other patterns

- Bridge
- Builder
- Chain of responsibility
- Flyweight
- Interpreter
- Mediator
- Memento
- Prototype
- Visitor

Bridge

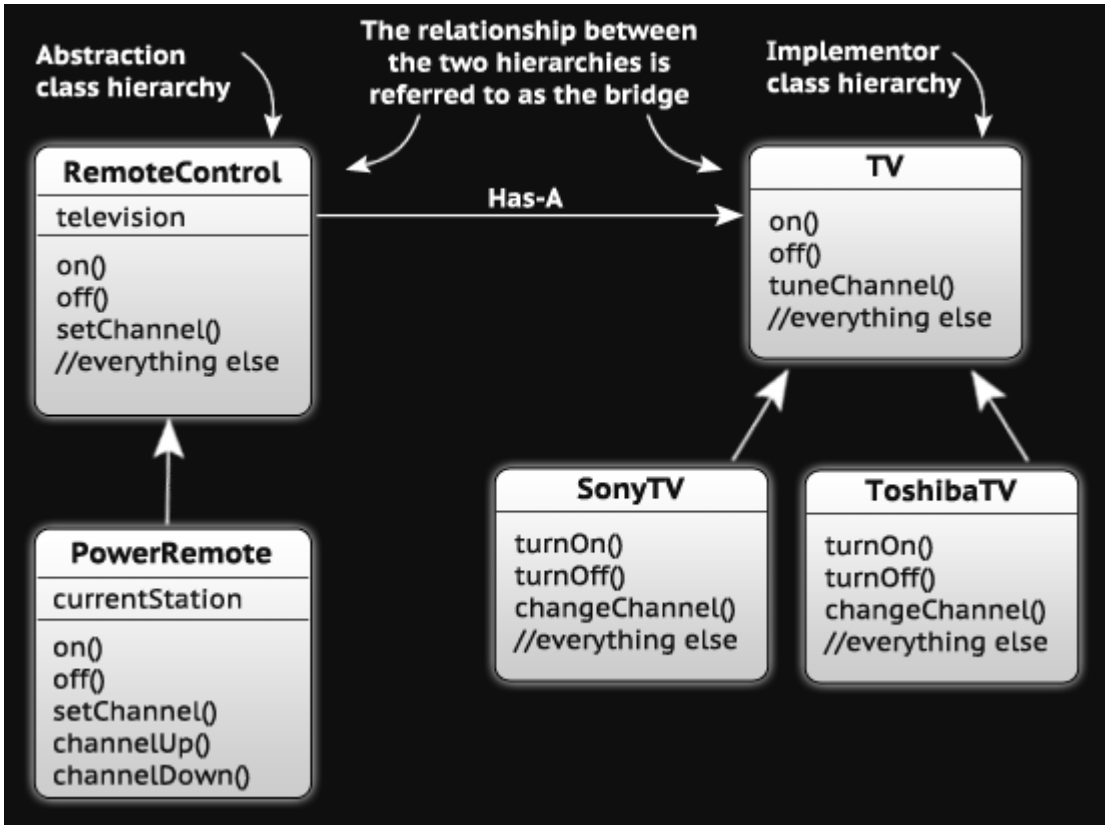
- Use the Bridge Pattern to vary not only your implementations, but also your abstractions.
- Scenario
  - You're writing the code for a new ergonomic and user-friendly remote control for TVs.
  - The remote is based on the same abstraction, but there will be lots of implementations—one for each model of TV.
- Your dilemma
  - The remotes are going to change and the TVs are going to change.
  - You've already abstracted the user interface so that you can vary the implementation over the many TVs your customers will own.
  - But you are also going to need to vary the abstraction because it is going to change over time as the remote is improved.
  - You need an OO design that allows you to vary the implementation and the abstraction

Bridge (2)

- The Bridge Pattern allows you to vary the implementation and the abstraction by placing the two in separate class hierarchies.

All methods in the abstraction are implemented in terms of the implementation

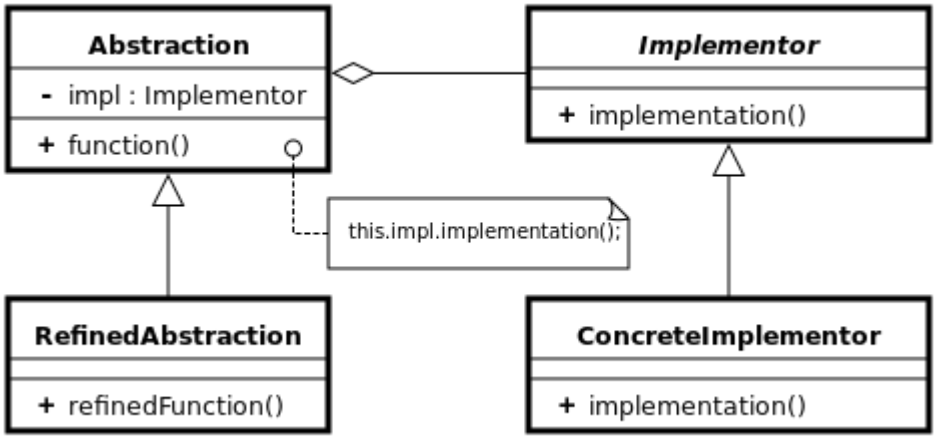
e.g. setChannel() =television.tuneChannel



Concrete subclasses are implemented in terms of the abstraction, not the implementation

The UML Model

All methods in the abstraction are implemented in terms of the implementation



Concrete subclasses are implemented in terms of the abstraction, not the implementation.

Example
TV as implementor

```
//Implementor
   public interface TV{
   public void on();
   public void off();
   public void tuneChannel(int channel);
}

//Concrete Implementor
public class Sony implements TV{
   public void on(){//Sony specific on}
   public void off(){//Sony specific off}
   public void tuneChannel(int channel);{//Sony specific tuneChannel}
}

//Concrete Implementor
public class Philips implements TV{
   public void on(){//Philips specific on}
   public void off(){//Philips specific off}
   public void tuneChannel(int channel);{//Philips specific tuneChannel}
}
```

Example
Remote as abstraction

```
//Abstraction
public abstract class RemoteControl{
   private TV implementor;
    public void on(){
         implementor.on();
    }
    public void off(){
         implementor.off();
    }
    public void setChannel(int channel){
         implementor.tuneChannel(channel);
    }
}
//Refined abstraction
public class ConcreteRemote extends RemoteControl{
    private int currentChannel;
    public void nextChannel(){
         currentChannel++;
         setChannel(currentChannel);
    }
    public void prevChannel(){
         currentChannel--;
         setChannel(currentChannel);
    }
```

Pros and Cons

- Decouples an implementation so that it is not bound permanently to an interface.
- Abstraction and implementation can be extended independently.
- Changes to the concrete abstraction classes don't affect the client.
- Useful in graphics and windowing systems that need to run over multiple platforms.
- Useful any time you need to vary an interface and an implementation in different ways.
- Increases complexity

The Builder pattern

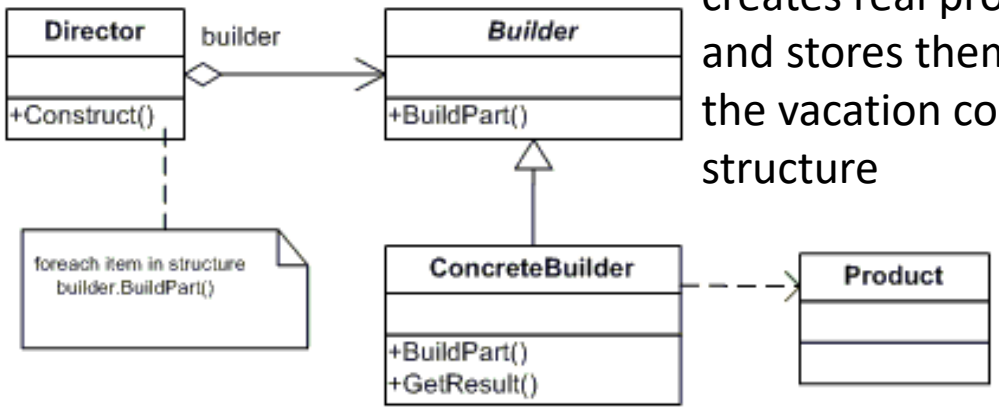
- Use the Builder Pattern to encapsulate the construction of a product and allow it to be constructed in steps.
- A scenario
  - An example of this is happy meals at a fast food restaurant. The happy meal typically consists of a hamburger, fries, coke and toy. No matter whether you choose different burgers/drinks, the construction of the kids meal follows the same process.
- So, you need
  - a flexible data structure that can represent happy meals and all their variations;
  - you also need to follow a sequence of potentially complex steps to create the happy meals.

The Builder pattern (2)

The Client directs the builder to construct the happy meal.

The client uses an abstract interface to build the planner.

The concrete builder creates real products and stores them in the vacation composite structure



In the iterator, we encapsulated the iteration into a separate object and hide the internal representation of the collection from the client. It's the same idea here: we encapsulate the creation of the happy meal in an object (let's call it a builder), and have our client ask the builder to construct the happy meal structure for it.

The Builder pattern - example

- The Builder

```
// Builder
public abstract class MealBuilder {
  protected Meal meal = new Meal();
  public abstract void buildDrink();
  public abstract void buildMain();
  public abstract void buildDessert();
  public abstract Meal getMeal();
}
```

The Builder pattern – example (2)

```
public class KidsMealBuilder extends MealBuilder {
    public void buildDrink() { // add drinks to the meal}
    public void buildMain() {// add main part of the meal}
    public void buildDessert() {// add dessert part to the meal}
    public Meal getMeal() {return meal;}
}
public class AdultMealBuilder extends MealBuilder {
    public void buildDrink(){// add drinks to the meal}
    public void buildMain(){// add main part of the meal}
public void buildDessert(){// add dessert part to the meal}
public Meal getMeal(){return meal;}
}
// Director
public class MealDirector {
    public Meal createMeal(MealBuilder builder) {
          builder.buildDrink();
          builder.buildMain();
          builder.buildDessert();
          return builder.getMeal();
}}
```

- We create two builders and one director

The Builder pattern – example (3)

- The client can change the builder

```
// Integration with overall application

public class Main {
        public static void main(String[] args) {
                MealDirector director = new MealDirector();
                MealBuilder builder = null;
                if (isKid) {
                        builder = new KidsMealBuilder();
                        }
                    else{
                        builder = new AdultMealBuilder();
                        }
                Meal meal = director.createMeal(builder);
        }
}
```

https://it.wikipedia.org/wiki/Builder

The Builder pattern – another example

```
/** "Product" */
class Pizza {
   private String dough = "";
   private String sauce = "";
   private String topping = "";

   public void setDough(String dough)
   { this.dough = dough; }
   public void setSauce(String sauce)
   { this.sauce = sauce; }
   public void setTopping(String topping)
   { this.topping = topping; }
}

/** "Abstract Builder" */
abstract class PizzaBuilder {
   protected Pizza pizza;

   public Pizza getPizza() {  return pizza;    }
   public void createNewPizzaProduct(){ pizza = new Pizza(); }

   public abstract void buildDough();
   public abstract void buildSauce();
   public abstract void buildTopping();
}
```

```
/** "ConcreteBuilder" */
class HawaiianPizzaBuilder extends PizzaBuilder
{
   public void buildDough()   {
      pizza.setDough("cross");
   }
   public void buildSauce()   {
      pizza.setSauce("mild");
   }
   public void buildTopping()   {
      pizza.setTopping("ham+pineapple");
   }
}
```

```
/** "ConcreteBuilder" */
class SpicyPizzaBuilder extends PizzaBuilder
{
   public void buildDough() {
      pizza.setDough("pan baked");
   }
   public void buildSauce(){
      pizza.setSauce("hot");
   }
   public void buildTopping() {
      pizza.setTopping("pepperoni+salami");
   }
}
```

The Builder pattern – another example

```
/** "Director" */
class Cook
{
   private PizzaBuilder pizzaBuilder;

   public void setPizzaBuilder(PizzaBuilder pb)
   {
      pizzaBuilder = pb;
   }
   public Pizza getPizza()
   {
      return pizzaBuilder.getPizza();
   }

   public void constructPizza()
   {
      pizzaBuilder.createNewPizzaProduct();
      pizzaBuilder.buildDough();
      pizzaBuilder.buildSauce();
      pizzaBuilder.buildTopping();
   }
}
```

```
/** A given type of pizza being constructed. */
class BuilderExample
{
   public static void main(String[] args)
   {
      Cook cook = new Cook();
      PizzaBuilder hawaiianPizzaBuilder = new
HawaiianPizzaBuilder();
      PizzaBuilder spicyPizzaBuilder = new SpicyPizzaBuilder();

      cook.setPizzaBuilder( hawaiianPizzaBuilder );
      cook.constructPizza();

      Pizza pizza = cook.getPizza();
   }
}
```

https://it.wikipedia.org/wiki/Builder

Builder pattern with nested classes

```
public class Person {
private String name;
private String email;
private int age;
private String address;

public static class Builder {
private final String name; // needed
private final String email; // needed
private int age;// optional
private String address;// optional

public Builder(String name, String email)
{
this.name = name;
this.email = email;
}

public Builder setAge(int age) {
this.age = age;
return this;
}

public Builder setAge(String address) {
this.address = address;
return this;
}

public Person build() {
return new Person(this);
}

}
```

```
public Person(Builder builder) {
this.name = builder.name;
this.email = builder.email;
this.age = builder.age;
this.address = builder.address;
}

@Override
public String toString() {
return this.name + " " + this.email + " " +
this.age + " " + this.address;
}

}
```

Pros and Cons

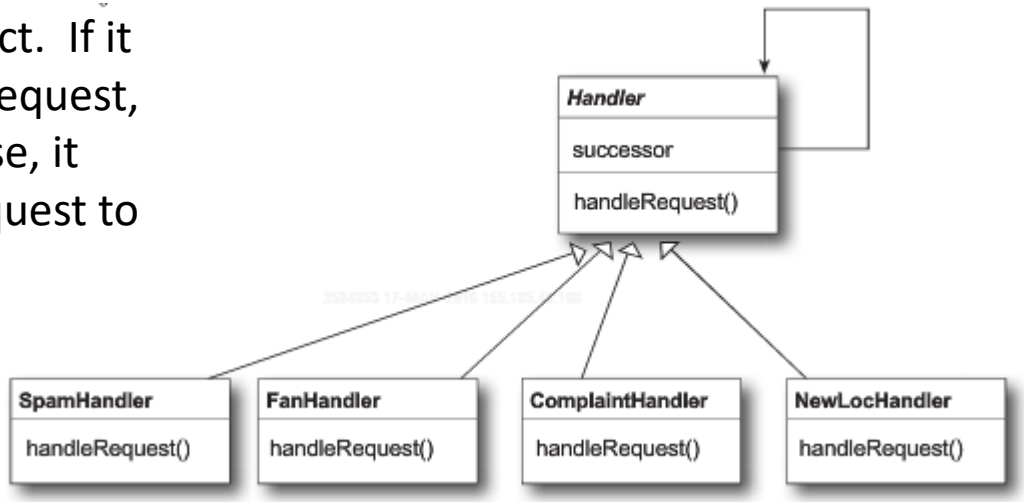
- Encapsulates the way a complex object is constructed.
- Allows objects to be constructed in a multistep and varying process (as opposed to one-step factories).
- Hides the internal representation of the product from the client.
- Product implementations can be swapped in and out because the client only sees an abstract interface.
- Often used for building composite structures.
- Constructing objects requires more domain knowledge of the client than when using a Factory.

Chain of Responsibility

- Use the Chain of Responsibility Pattern when you want to give more than one object a chance to handle a request.
- Scenario
  - A company has been getting more email than they can handle. From their own analysis they get four kinds of email:
    - fan mails -- > go straight to the CEO
    - complaints -- > go to the legal department
    - requests -- >go to business development
    - Spam -- > deleted
- The task
  - The company has already written some AI detectors that can tell the kind of the e-mail, but they need you to create a design that can use the detectors to handle incoming email.

Chain of Responsibility (2)

Each object in the chain acts as a handler and has a successor object. If it can handle the request, it does; otherwise, it forwards the request to its successor.



- As email is received, it is passed to the first handler: the SpamHandler. If the SpamHandler can't handle the request, it is passed on to the FanHandler. And so on..

Example

```
abstract class PurchasePower {
protected static final double BASE = 500;
protected PurchasePower successor;
abstract protected double getAllowable();
abstract protected String getRole();

public void setSuccessor(PurchasePower successor) {
                this.successor = successor; }

public void processRequest(PurchaseRequest request){
                if (request.getAmount() < this.getAllowable()) {
                System.out.println(this.getRole() + " will approve $" + request.getAmount()); }
                 else if (successor != null) { successor.processRequest(request); }
                }
 }
```

https://it.wikipedia.org/wiki/Chain-of-responsibility_pattern

Example

```
class ManagerPPower extends PurchasePower {

   protected double getAllowable(){
      return BASE*10;
   }

   protected String getRole(){
      return "Manager";
   }
}

class DirectorPPower extends PurchasePower {

   protected double getAllowable(){
      return BASE*20;
   }

   protected String getRole(){
      return "Director";
   }
}
```

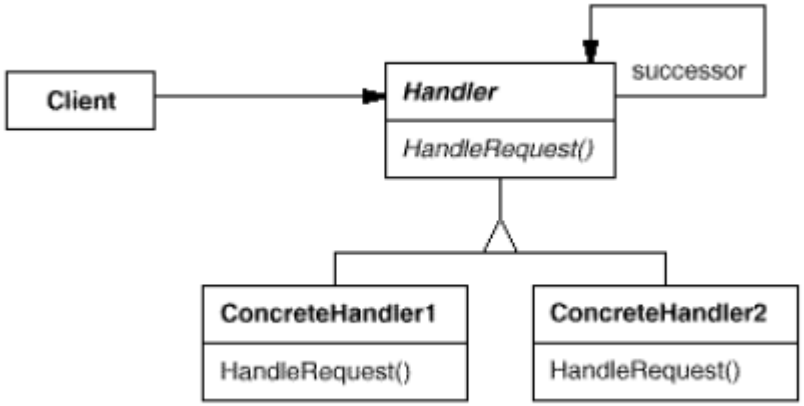
```
class VicePresidentPPower extends PurchasePower
{

   protected double getAllowable(){
      return BASE*40;
   }

   protected String getRole(){
      return "Vice President";
   }
}

class PresidentPPower extends PurchasePower {

   protected double getAllowable(){
      return BASE*60;
   }

   protected String getRole(){
      return "President";
   }
}
```

Example

```
class PurchaseRequest {
   private double amount;
   private String purpose;

   public PurchaseRequest(double amount, String
purpose) {
      this.amount = amount;
      this.purpose = purpose;
   }

   public double getAmount() {
      return amount;
   }
   public void setAmount(double amt)  {
      amount = amt;
   }

   public String getPurpose() {
      return purpose;
   }
   public void setPurpose(String reason) {
      purpose = reason;
   }
}
```

Example

```
class CheckAuthority {
   public static void main(String[] args) {
      ManagerPPower manager = new ManagerPPower();
      DirectorPPower director = new DirectorPPower();
      VicePresidentPPower vp = new VicePresidentPPower();
      PresidentPPower president = new PresidentPPower();
      manager.setSuccessor(director);
      director.setSuccessor(vp);
      vp.setSuccessor(president);

      // Press Ctrl+C to end.
      try {
         while (true) {
            System.out.println("Enter the amount to check who should approve your expenditure.");
            System.out.print(">");
            double d = Double.parseDouble(new BufferedReader(new
InputStreamReader(System.in)).readLine());
            manager.processRequest(new PurchaseRequest(d, "General"));
         }
      } catch(Exception e) {
         System.exit(1);
      }
   }
}
```



Pros and Cons

- Decouples the sender of the request and its receivers.
- Simplifies your object because it doesn't have to know the chain's structure and keep direct references to its members.
- Allows you to add or remove responsibilities dynamically by changing the members or order of the chain.
- Commonly used in windows systems to handle events like mouse clicks and keyboard events.
- Execution of the request isn't guaranteed; it may fall off the end of the chain if no object handles it (this can be an advantage or a disadvantage).
- Can be hard to observe and debug at runtime.

Flyweight

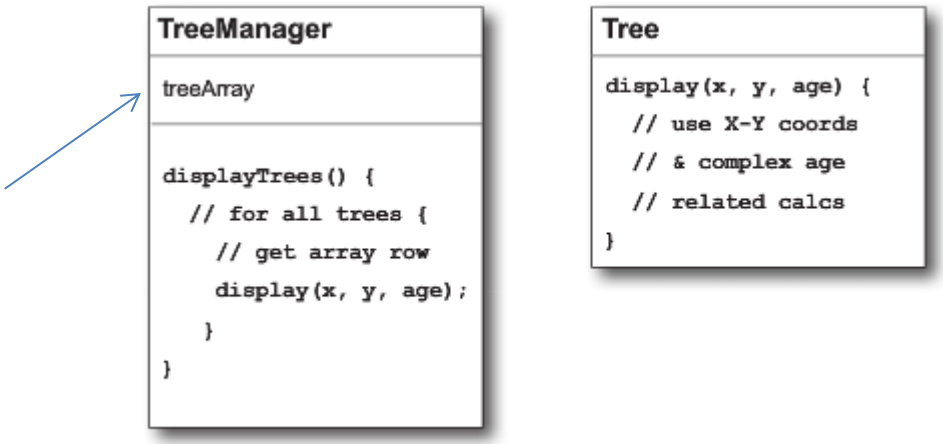
- Use the Flyweight Pattern when one instance of a class can be used to provide many “virtual instances.”
- A scenario
  - You want to add trees as objects in your hot new landscape design application. In your application, trees don’t really do very much; they have an X-Y location, and they can draw themselves dynamically, depending on how old they are. The thing is, a user might want to have lots and lots of trees in one of their home landscape designs.
- Your big client’s dilemma
  - You’ve just landed your “reference account.” That key client you’ve been pitching for months. They’re going to buy 1,000 seats of your application, and they’re using your software to do the landscape design for huge planned communities. After using your software for a week, your client is complaining that when they create large groves of trees, the app starts getting sluggish...

ONLY FOR YOUR REFERENCE

Flyweight (2)

- What if, instead of having thousands of Tree objects, you could redesign your system so that you've got only one instance of Tree, and a client object that maintains the state of ALL your trees.

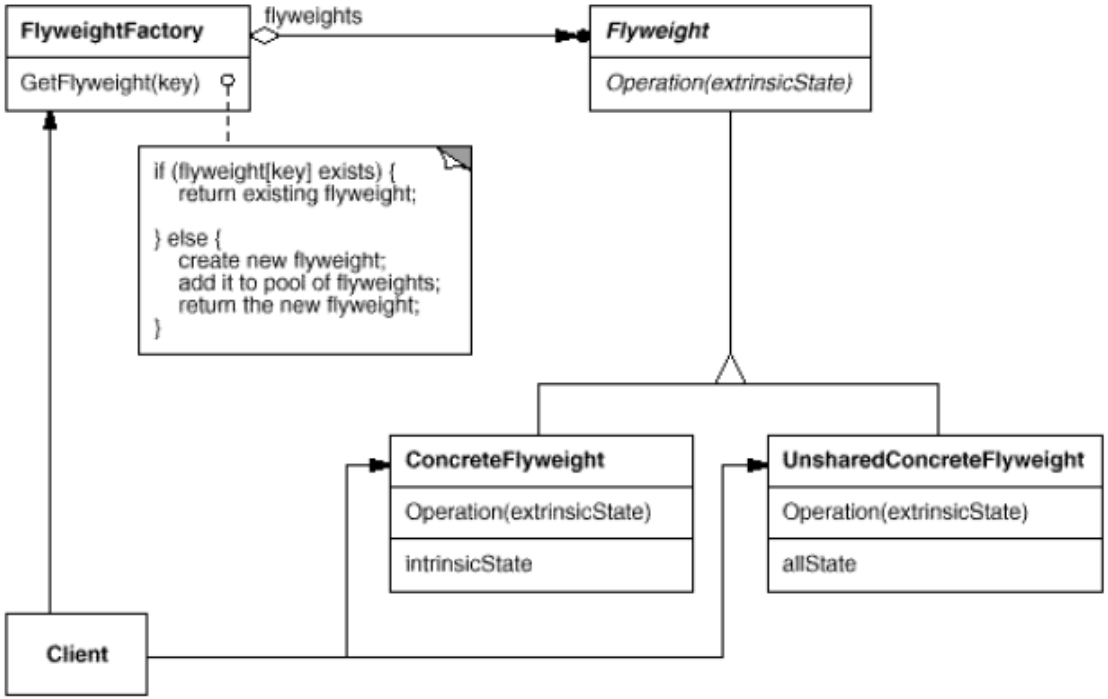
All the state, for ALL of your virtual Tree objects, is stored in this 2D-array.



ONLY FOR YOUR REFERENCE

Flyweight (3)

Actually the GoF Model is a little bit more complex.



ONLY FOR YOUR REFERENCE

Pros and Cons

- Reduces the number of object instances at runtime, saving memory.
- Centralizes state for many “virtual” objects into a single location.
- The Flyweight is used when a class has many instances, and they can all be controlled identically.
- A drawback of the Flyweight pattern is that once you’ve implemented it, single, logical instances of the class will not be able to behave independently from the other instances.

ONLY FOR YOUR REFERENCE

Interpreter

- Use the Interpreter Pattern to build an interpreter for a language.
- The Interpreter Pattern requires some knowledge of formal grammars.
- Representing each grammar rule in a class makes the language easy to implement.
- Because the grammar is represented by classes, you can easily change or extend the language.
- By adding methods to the class structure, you can add new behaviors beyond interpretation, like pretty printing and more sophisticated program validation.

ONLY FOR YOUR REFERENCE

Mediator

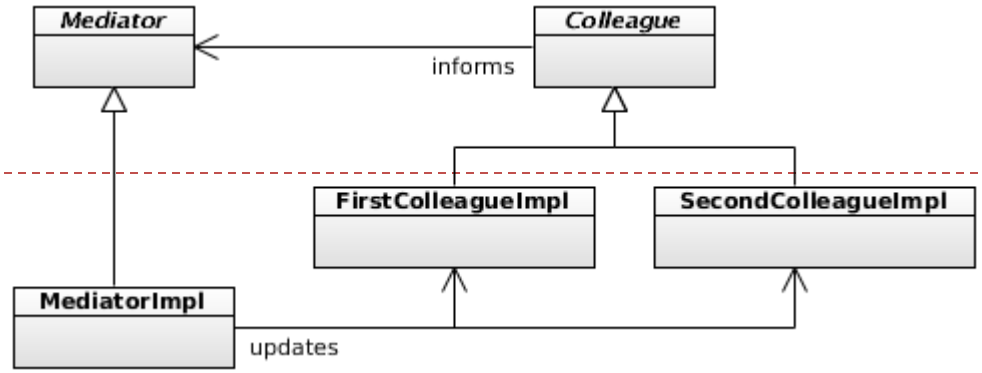
- Use the Mediator Pattern to centralize complex communications and control between related objects.
- A scenario
  - Bob has a Java-enabled auto-house. All of his appliances are designed to make his life easier. When Bob stops hitting the snooze button, his alarm clock tells the coffee maker to start brewing.
  - Even though life is good for Bob, he and other clients are always asking for lots of new features: No coffee on the weekends... Turn off the sprinkler 15 minutes before a shower is scheduled... Set the alarm early on trash days...
- It's getting really hard to keep track of which rules reside in which objects, and how the various objects should relate to each other.

ONLY FOR YOUR REFERENCE

Mediator (2)

- With a Mediator added to the system, all of the appliance objects can be greatly simplified:
  - They tell the Mediator when their state changes.
  - They respond to requests from the Mediator.
- Before we added the Mediator, all of the appliance objects needed to know about each other... they were all tightly coupled. With the Mediator in place, the appliance objects are all completely decoupled from each other.
- The Mediator contains all of the control logic for the entire system. When an existing appliance needs a new rule, or a new appliance is added to the system, you'll know that all of the necessary logic will be added to the Mediator.

ONLY FOR YOUR REFERENCE



Pros and Cons

- Increases the reusability of the objects supported by the Mediator by decoupling them from the system.
- Simplifies maintenance of the system by centralizing control logic.
- Simplifies and reduces the variety of messages sent between objects in the system.
- The Mediator is commonly used to coordinate related GUI components.
- A drawback of the Mediator Pattern is that without proper design, the Mediator object itself can become overly complex.

ONLY FOR YOUR REFERENCE

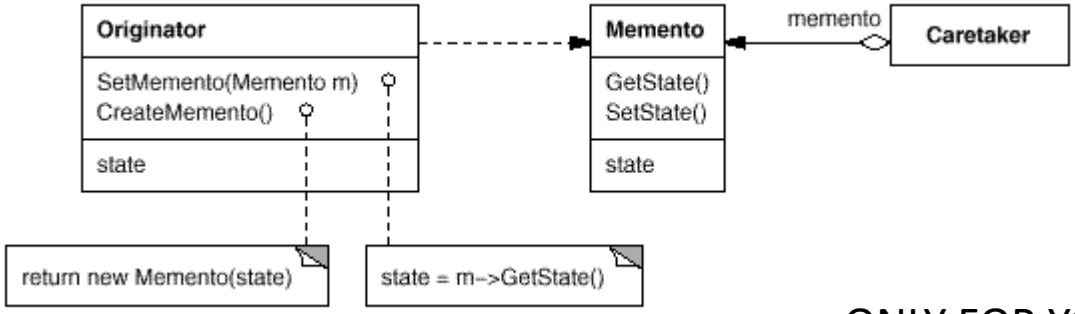
Memento

- Use the Memento Pattern when you need to be able to return an object to one of its previous states; for instance, if your user requests an “undo.”

ONLY FOR YOUR REFERENCE

Memento (2)

- The Memento has two goals:
  - Saving the important state of a system's key object.
  - Maintaining the key object's encapsulation.
- Keeping the single responsibility principle in mind, it's also a good idea to keep the state that you're saving separate from the key object. This separate object that holds the state is known as the Memento object.



ONLY FOR YOUR REFERENCE

Pros and Cons

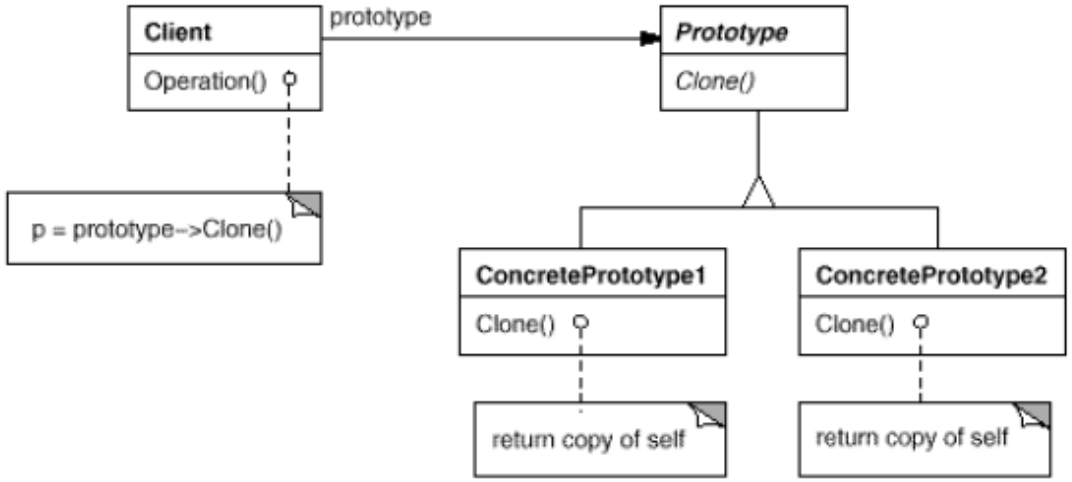
- Keeping the saved state external from the key object helps to maintain cohesion.
- Keeps the key object's data encapsulated.
- Provides easy-to-implement recovery capability.
- The Memento is used to save state.
- A drawback to using Memento is that saving and restoring state can be time consuming.
- In Java systems, consider using Serialization to save a system's state.

ONLY FOR YOUR REFERENCE

Prototype

- Use the Prototype Pattern when creating an instance of a given class is either expensive or complicated.
- A scenario
  - Your interactive role playing game has an insatiable appetite for monsters. As your heroes make their journey through a dynamically created landscape, they encounter an endless chain of foes that must be subdued. You'd like the monster's characteristics to evolve with the changing landscape. It doesn't make a lot of sense for bird-like monsters to follow your characters into underseas realms. Finally, you'd like to allow advanced players to create their own custom monsters.
- The Prototype Pattern allows you to make new instances by copying existing instances.
  - In Java this typically means using the clone() method, or de-serialization when you need deep copies.
- A key aspect of this pattern is that the client code can make new instances without knowing which specific class is being instantiated.

ONLY FOR YOUR REFERENCE



Pros and Cons

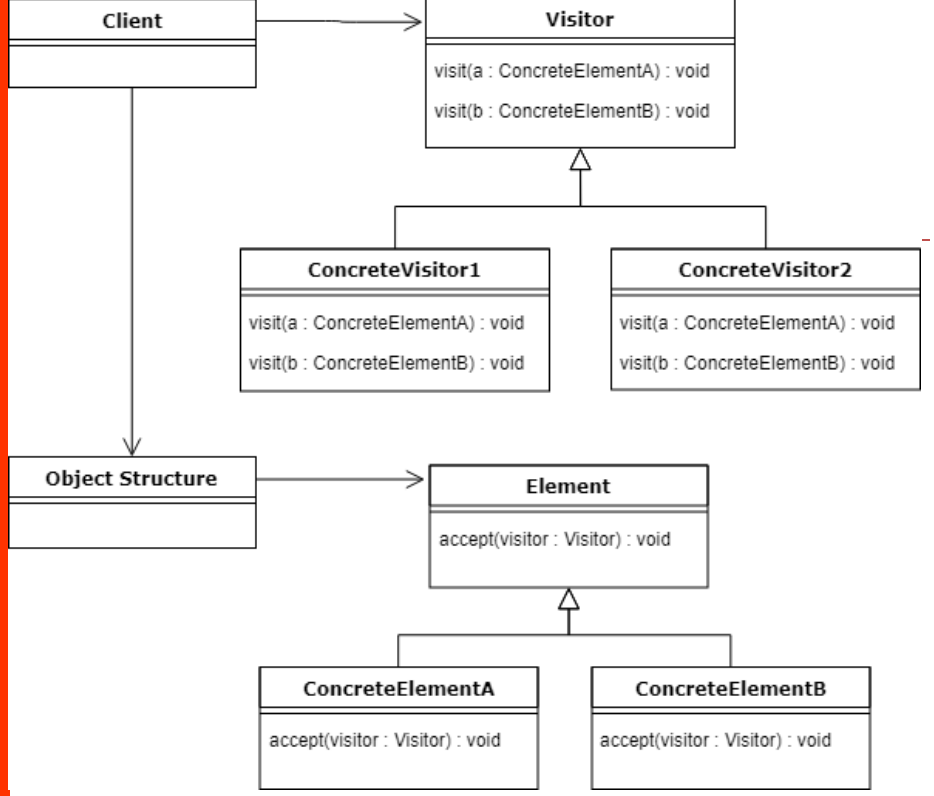
- Hides the complexities of making new instances from the client.
- Provides the option for the client to generate objects whose type is not known.
- In some circumstances, copying an object can be more efficient than creating a new object.
- Prototype should be considered when a system must create new objects of many types in a complex class hierarchy.
- A drawback to using the Prototype is that making a copy of an object can sometimes be complicated.

https://www.geeksforgeeks.org/visitor-design-pattern/

Visitors

- It is used when we have to perform an operation on a group of similar kind of Objects. We can move the operational logic from the objects to another class.
- The visitor pattern consists of two parts:
  - a method **Visit()** implemented by the visitor and called for every element in the data structure
  - visitable classes providing **Accept()** methods that accept a visitor

Visitors



- **Client :** The Client class is a consumer of the classes of the visitor design pattern. It has access to the data structure objects and can instruct them to accept a Visitor to perform the appropriate processing.
- **Visitor :** This is an interface or an abstract class used to declare the visit operations.
- **ConcreteVisitor :** For each type of visitor all the visit methods, declared in abstract visitor, must be implemented. Each Visitor will be responsible for different operations.
- **Visitable :** This is an interface which declares the accept operation. This is the entry point which enables an object to be “visited” by the visitor object.
- **ConcreteVisitable :** These classes implement the Visitable interface or class and defines the accept operation. The visitor object is passed to this object using the accept operation.

Visitors

- Use the Visitor Pattern when you want to add capabilities to a composite of objects and encapsulation is not important.
- Another scenario
  - Customers who frequent a restaurant have recently become more health conscious. They are asking for nutritional information before ordering their meals. Some customers are even asking for nutritional information on a per ingredient basis.
- The Visitor works hand in hand with a Traverser.
  - The Traverser knows how to navigate to all of the objects in a Composite.
  - The Traverser guides the Visitor through the Composite so that the Visitor can collect state as it goes.
  - Once state has been gathered, the Client can have the Visitor perform various operations on the state.
  - When new functionality is required, only the Visitor must be enhanced.

Pros and Cons

- Allows you to add operations to a Composite structure without changing the structure itself.
- Adding new operations is relatively easy.
- The code for operations performed by the Visitor is centralized.
- The Composite classes' encapsulation is broken when the Visitor is used.
- Because the traversal function is involved, changes to the Composite structure are more difficult.

Patterns in the real world

- A Pattern is a solution to a problem in a context.
  - The context is the situation in which the pattern applies. This should be a recurring situation.
  - The problem refers to the goal you are trying to achieve in this context, but it also refers to any constraints that occur in the context.
  - The solution is what you're after: a general design that anyone can apply which resolves the goal and set of constraints.

Patterns in the real world

- Is it possible to create your own Design Patterns? Or is that something you have to be a “patterns guru” to do?
- Patterns are discovered, not created.
  - Anyone can discover a Design Pattern and then author its description; however, it’s not easy and doesn’t happen quickly, nor often. Being a “patterns writer” takes commitment.
  - You might work in a specialized domain for which you think new patterns would be helpful, or you might have come across a solution to what you think is a recurring problem, or you may just want to get involved in the patterns community and contribute to the growing body of work.
- You don’t have a pattern until others have used it and found it to work.
  - You don’t have a pattern until it passes the “Rule of Three.”
    - This rule states that a pattern can be called a pattern only if it has been applied in a real-world solution at least three times.

So you want to be a Design Patterns writer

- Do your homework.
  - You need to be well versed in the existing patterns before you can create a new one. Most patterns that appear to be new, are, in fact, just variants of existing patterns
- Take time to reflect, evaluate.
  - Your experience—the problems you've encountered, and the solutions you've used —are where ideas for patterns are born.
- Get your ideas down on paper in a way others can understand.
  - Locating new patterns isn't of much use if others can't make use of your find; you need to document your pattern candidates so that others can read, understand, and apply them to their own solution and then supply you with feedback. Use the GoF template.
- Have others try your patterns; then refine and refine some more.
- Have other developers review your candidate pattern, try it out, and give you feedback.
  - Incorporate that feedback into your description and try again.
- Don't forget the Rule of Three.

References

- Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, O'Reilly Media 2004
- Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994
- https://dzone.com/articles/*